2.3 The CQP interface

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 21 October 2026, 13:37

This supplementary handout introduces the CQP query syntax used on corpus platforms such as CQPweb, covering queries for word forms, lemmas and part-of-speech tags, sequences of tokens, the main differences to regular expressions in quanteda, and how to export the results for further analysis in R.

Table of contents

Introduction 1
Basic use 2
Searching for sequences of tokens 3
CQP vs. regular expressions in R 4
Exporting and importing your results 4
    Option 1: Directly into R 5
    Option 2: Via Excel 5
Exercises 5

Introduction

Some corpus platforms such as BNCweb, CQPweb or our local KU corpora[1] support a specialised query syntax known as **CQP** (**C**orpus **Q**uery **P**rocessor). It enables users to search not only for word forms, but also for lemmas, part-of-speech (POS) tags and sequences of tokens, and to restrict their searches to texts with specific meta-attributes (text category, age, gender etc.).

[1]Note that the corpus platform of the Catholic University of Eichstätt-Ingolstadt, which was set up by Dr. Thomas Brunner, is only accessible to students or staff via the local **eduroam** network or a **VPN client** which holds their credentials.

Basic use

Once signed in on any of these platforms, the user interface will generally follow this layout:

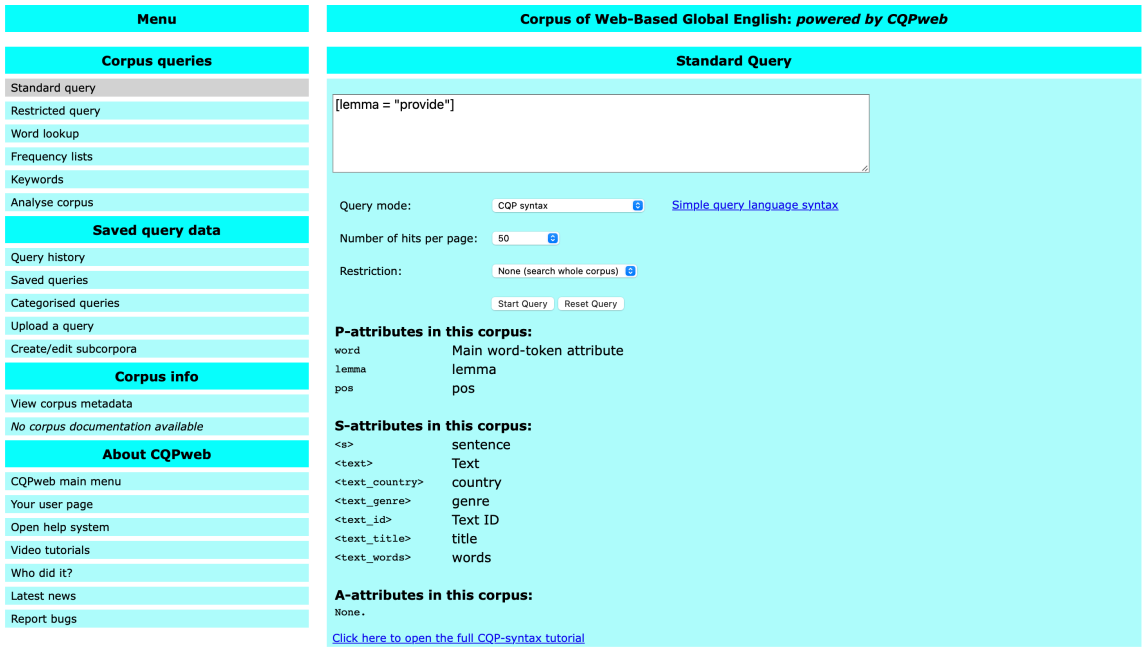


Figure 1: CQP query of PROVIDE in the GloWbE corpus

- To select a different corpus, navigate to `CQPweb main menu` under **About CQPweb** in the left menu bar.
- To restrict search results to specific parts of the corpus, select `Restricted query` and choose the relevant text categories, varieties, etc.

> **!** Switch the query mode
>
> By default, CQPweb interprets your input as a `Simple query`. To use the syntax introduced below, select `CQP syntax` in the `Query mode` drop-down menu next to the search box. Otherwise, your square brackets will not be interpreted as intended.

You can now enter a search expression into the white box. Each token you are looking for is described by a pair of square brackets containing an **attribute** and its **value**:

```
[attribute = "value"]
```

For example, to retrieve all inflectional forms of the verb *provide*, enter:

```
[lemma = "provide"]
```

The available attributes and their names depend on the corpus. Some corpora call the lemma attribute `hw` ('headword', e.g., BNCweb), while others do not support lemma searches at all.[2] You can usually find the available attributes on the corpus information page.

To search for a word with a specific part-of-speech (POS) tag, such as *like* used as a preposition, combine two attributes with `&` ('and'):

```
[word = "like" & pos = "ii"]
```

For a full list of POS tags, refer to:

- CLAWS5 (for **BNC**)
- CLAWS7 (for **COCA** and **GloWbE**)

Note that the two tagsets differ: In the BNC (CLAWS5), *like* as a preposition is tagged `PRP` rather than `II`. Furthermore, some corpora store their tags in lower case (e.g., `ii`, as in the GloWbE version used here), others in upper case (e.g., `II`).

Searching for sequences of tokens

If the exact word form is not important, the `word` attribute can simply be omitted. For example, to find all singular common nouns (`nn1`) that are followed by the preposition *of* (`io`), we can write

```
[pos = "nn1"][pos = "io"]
```

The values of an attribute are interpreted as regular expressions, which allows us to match multiple POS tags at once, such as all noun categories (`nn`, `nn1`, `nn2`, `np1` etc.) or all prepositions (`if`, `ii`, `io`, `iw`). The search term below generalises the previous example, making it more inclusive:

```
[pos = "n.+"][pos = "i.+"]
```

However, be cautious with overly broad search patterns, as they might return (nearly) the entire corpus! In GloWbE, the last query yields 22,294,282 matches.

An empty pair of square brackets `[]` matches **any** token. Combined with a quantifier in curly brackets, it allows us to search for words that occur within a certain distance of each other. The following query finds all forms of *provide* followed by *with* after zero to three intervening tokens (e.g., *provide* ***with***, *provided* ***them with***, *provides* ***the students with***):

[2] In that case, all inflectional forms have to be listed by means of a regular expression, such as `[word = "provid(e[sd]?|ing)"]` for *provide*.

```
[lemma = "provide"] []{0,3} [word = "with"]
```

CQP vs. regular expressions in R

If you have worked with `kwic()` (see 2.1 Concordancing and 2.2 Regular expressions), keep in mind the following differences:

Table 1: Querying with CQP and `kwic()`

	CQP	`kwic()` in `quanteda`
Matching	The regular expression must match the **entire** token: `[word = "eat"]` finds *eat*, but not *eats* or *eating*.	The regular expression may match **any part** of a token unless it is anchored with `^` and `$`.
Case	**Case-sensitive** by default. Add `%c` after the value to ignore case: `[word = "like"%c]`.	**Case-insensitive** by default.
Annotation	Lemmas, POS tags and other attributes can be queried directly (if the corpus provides them).	Only the word forms are available, unless the corpus has been tagged beforehand.
Sequences	One pair of brackets per token, with `[]` as a placeholder.	Multi-word patterns require `phrase()`.

For instance, the CQP equivalent of the `kwic()` query for all forms of *eat* in 2.4 Data annotation is

```
[word = "(eat(s|ing|en)?|ate)"%c]
```

which needs neither `^` nor `$`, but `%c` to also find *Eat* or *Eating* at the beginning of a sentence.

Exporting and importing your results

To export your KWIC hits, select `Download` in the top-right corner and specify your output options (e.g., the size of the search window or the speaker metadata). CQPweb produces a tab-delimited text file (e.g., `concordance.txt`), which can be imported in two ways.

Option 1: Directly into R

The most reliable option is to read the file directly into R and then export it as an Excel file for annotation (see 1.6 Importing/Exporting):

```
library(readr)
library(writexl)

# Read the tab-delimited file; quote = "" prevents quotation marks
# in the concordance lines from being misinterpreted
concordance <- read_tsv("concordance.txt", quote = "")

# Check whether all rows and columns have been read in correctly
head(concordance)

# Export as an Excel file for annotation
write_xlsx(concordance, "concordance.xlsx")
```

Depending on the download options, the file may begin with a few lines of information about the query. If the column names look odd, open the file in a text editor and skip these lines with the `skip` argument, e.g., `read_tsv("concordance.txt", quote = "", skip = 3)`.

Option 2: Via Excel

Alternatively, you can import the file in spreadsheet software and perform some **manual clean-up** to ensure that all columns and rows are complete and properly formatted. In MS Excel:

1. Navigate to `Data > From Text/CSV` (Windows) or `File > Import > Text file` (Mac).
2. Select your file and choose `Delimited`.
3. Select `Tab` as the delimiter.
4. Finish the import.

Make sure to **save your file** with the extension `.xlsx`. Be aware that Excel occasionally converts values automatically (e.g., strings such as *1-2* into dates). It is therefore worth checking the imported data carefully.

Exercises

Exercise 1. Write a CQP query that finds all forms of *provide* followed by the preposition *with* within a distance of up to five tokens. How many of the hits are instances of the construction *provide somebody with something*?

Exercise 2. The word *like* can be used as a preposition (*a day like this*) or as a verb (*I like tea*). Look up the relevant CLAWS7 tags and write two queries that distinguish between these uses. Which one is more frequent?

Exercise 3. Explain why the query `[word = "eat"]` finds fewer hits than `[word = "eat"%c]`, and why `[word = "eat.*"]` would be too inclusive.